

CS101: Software Engineering Principles

Week 3: Breaking Problems Down

Milestone 1: Conceptual Scoping & Problem Engineering

Before writing a single line of production code, a software engineer must establish absolute clarity regarding why a feature exists and what boundary constraints it must satisfy to prevent system scope creep.

Lesson 1: Writing a Clear Feature Brief

A professional Feature Brief is a tight, 4–6 sentence paragraph explaining a feature's scope without leaking technical implementation details. Every high-quality brief uses the 1-2-3 Drafting Method to guarantee three core dimensions are accounted for:

■ Click to Expand: The 1-2-3 Drafting Breakdown

- **Goal:** The direct business value or user problem the code solves.
- **Target Users:** Exactly who will be interacting with this system space.
- **Context:** The concrete situation, time of day, or environment where execution occurs.

Milestone 2: Data Routing Contracts & Invariant Testing

Lesson 4: Defining Example Inputs and Outputs

Every subproblem requires an input/output map. When constructing these tables, you must explicitly account for Normal Cases along with boundary-pushing Edge Cases where processing errors typically manifest.

Lesson 12: Writing a Full Problem Statement with Assertions

Instead of using visual screen print statements to manually inspect behavior, professional environments use assertions to enforce hard constraints and protect calculation invariants.

```
def calculate_tip(bill, tip_percent, num_people):  
    tip_amount = round(bill * (tip_percent / 100), 2)  
    grand_total = round(bill + tip_amount, 2)  
    per_person = round(grand_total / num_people, 2)  
    return tip_amount, grand_total, per_person
```

Milestone 3: Functional Decomposition & Structural Patterns

Lesson 8: Matching Subproblems to Patterns

To lower your cognitive overhead when turning a plan into logic, map the active verb of your subproblem directly onto a structural Python layout pattern:

Subproblem Core Verb	Target Pattern Name	Code Hint Style
look up / find / translate	Lookup Table	<code>value = map.get(key, default)</code>
calculate total / collect	Accumulator Loop	<code>total += score</code>
skip bad data	Filter Loop	<code>if invalid: continue</code>
exit on bad input	Guard Clause	<code>if not data: return</code>